

Corrigé de la série 4: Factory/Observer/Chain of Responsibility

Mise à jour avril
2023

Exercice 1

On voudrais créer un gestionnaire pour des demandes d'authentification, la méthode booléenne `isAuthorized()` est celle qui gère l'autorisation, on aura deux manières pour s'authentifier : username et mot de passe et le protocole de délégation d'autorisation OAuth.

Solution exercice 1

```
public abstract class AuthenticationProcessor {

    public AuthenticationProcessor nextProcessor;

    // standard constructors

    public abstract boolean isAuthorized(AuthenticationProvider
authProvider);
}
.....le 1er gestionnaires concret.....
public class OAuthProcessor extends AuthenticationProcessor {

    public OAuthProcessor(AuthenticationProcessor nextProcessor) {
        super(nextProcessor);
    }

    @Override
    public boolean isAuthorized(AuthenticationProvider authProvider) {
        if (authProvider instanceof OAuthTokenProvider) {
            return true;
        } else if (nextProcessor != null) {
            return nextProcessor.isAuthorized(authProvider);
        }

        return false;
    }
}
..... le 2eme gestionnaires concret.....

public class UsernamePasswordProcessor extends AuthenticationProcessor {

    public UsernamePasswordProcessor(AuthenticationProcessor nextProcessor)
{
        super(nextProcessor);
    }

    @Override
    public boolean isAuthorized(AuthenticationProvider authProvider) {
        if (authProvider instanceof UsernamePasswordProvider) {
            return true;
        } else if (nextProcessor != null) {
            return nextProcessor.isAuthorized(authProvider);
        }
    }
    return false;
}
..... la classe test.....

Public class CORTest{
main(){

//on peut créer et affecter les successeurs dans la meme instruction
AuthenticationProcessor oAuthProcessor = new OAuthProcessor(null);
AuthenticationProcessor usernamePasswordProcessor =new
UsernamePasswordProcessor(oAuthProcessor);
```

```

//on déclenche l'authentification avec le 2me gestionnaire
// OAuthTokenProvider est une classe predefinie où sont sauvegardés des
informations de connexion.
usernamePasswordProcessor.isAuthenticated(new OAuthTokenProvider());
//on déclenche l'authentification une 2me fois avec un systeme d'auth:

// SamlTokenProvider est une classe predefinie où l'auth est gérée grace au
protocole Saml
usernamePasswordProcessor.isAuthenticated(new SamlTokenProvider());

}}
comme résultat d'exécution; on aura true pour le 1er appel, false pour le
second, car Saml n'a pas été introduit dans la chaine de responsabilité

+++++ce qui suit est une autre façon de faire des test avec
l'outil JUnit de Java, seulement pour les intéressés+++++

public class ChainOfResponsibilityTest {

    private static AuthenticationProcessor getChainOfAuthProcessor() {
        AuthenticationProcessor oAuthProcessor = new OAuthProcessor(null);
        return new UsernamePasswordProcessor(oAuthProcessor);
    }

    @Test
    public void givenOAuthProvider_whenCheckingAuthorized_thenSuccess()
{AuthenticationProcessor authProcessorChain = getChainOfAuthProcessor();
    assertTrue(authProcessorChain.isAuthenticated(new OAuthTokenProvider()));
    }

    @Test
    public void givenSamlProvider_whenCheckingAuthorized_thenSuccess() {
        AuthenticationProcessor authProcessorChain =
getChainOfAuthProcessor();

        assertFalse(authProcessorChain.isAuthenticated(new SamlTokenProvider()));
    }
}

```

Exercice 2

On voudrait implémenter un distributeur de billets (ATM dispense machine) ; l'utilisateur entre le montant qu'il voudrais avoir et la machine lui remet le montant sous forme de billets de 1000DA 500DA 200DA. Si l'utilisateur fait entrer un montant non multiple de dix, la machine déclare une erreur.

Solution exercice 2

+++++=la classe qui renvoie le montant **Currency** ++++++

```
public class Currency {  
  
    private int amount;  
  
    public Currency(int amt){  
        this.amount=amt;  
    }  
  
    public int getAmount(){  
        return this.amount;  
    }  
}
```

+++++l'interface de gestion **DispenseChain** (abstract Handler, cela peut être une interface au prix d'initialiser le successeur dans chaque classe d'implémentation (sous classe))+++++

```
public interface DispenseChain {  
  
    void setNextChain(DispenseChain nextChain);  
  
    void dispense(Currency cur);  
}
```

+++++une des classes de gestion concrete **DA500Dispenser** (dispense les billets de 500 DA)+++++

```
public class DA500Dispenser implements DispenseChain {  
private DispenseChain chain;  
@Override  
    public void setNextChain(DispenseChain nextChain) {  
        this.chain=nextChain;  
    }  
  
    @Override  
    public void dispense(Currency cur) {  
        if(cur.getAmount() >= 500){  
            int num = cur.getAmount()/500;  
            int remainder = cur.getAmount() % 500;  
            System.out.println("Dispensing "+num+" 500DA note");  
            if(remainder !=0) this.chain.dispense(new Currency(remainder));  
        }else{  
            this.chain.dispense(cur);  
        }  
    }  
}
```

+++++++la deuxième classe de gestion concrete **DA200Dispenser** (dispense les billets de 200 DA++++++

```
package com.journaldev.design.chainofresponsibility;
```

```
public class DA200Dispenser implements DispenseChain{
```

```
    private DispenseChain chain;
```

```
    @Override
```

```
    public void setNextChain(DispenseChain nextChain) {  
        this.chain=nextChain;  
    }
```

```
    @Override
```

```
    public void dispense(Currency cur) {  
        if(cur.getAmount() >= 200){  
            int num = cur.getAmount()/200;  
            int remainder = cur.getAmount() % 200;  
            System.out.println("Dispensing "+num+" 200DA note");  
            if(remainder !=0) this.chain.dispense(new Currency(remainder));  
        }else{  
            this.chain.dispense(cur);  
        }  
    }
```

Attention, ici s'il reste de la monnaie, on doit déclarer le montant demandé insolvable!

```
}
```

+++++++la troisième classe de gestion concrete **DA1000Dispenser** (dispense les billets de 1000 DA++++++

```
public class DA1000Dispenser implements DispenseChain {
```

```
    private DispenseChain chain;
```

```
    @Override
```

```
    public void setNextChain(DispenseChain nextChain) {  
        this.chain=nextChain;  
    }
```

```
    @Override
```

```
    public void dispense(Currency cur) {  
        if (cur.getAmount() >= 1000){  
            int num = cur.getAmount()/1000;  
            int remainder = cur.getAmount() % 1000;  
            System.out.println("Dispensing "+num+" 1000DA note");  
            if(remainder !=0) this.chain.dispense(new Currency(remainder));  
        }else {  
            this.chain.dispense(cur);  
        }  
    }
```

+++++++la classe de test ditributeur de billets **ATMDispenseChain**+++++++

```
import java.util.Scanner;
```

```
public class ATMDispenseChain {
```

```
    private DispenseChain c1;
```

```
    public ATMDispenseChain() {  
        // initialiser la chaine  
        this.c1 = new DA1000Dispenser();  
        DispenseChain c2 = new DA500Dispenser();  
        DispenseChain c3 = new DA2000Dispenser();
```

```
        // établir la chaine de responsabilité  
        c1.setNextChain(c2);  
        c2.setNextChain(c3);  
    }
```

```
    public static void main(String[] args) {  
        ATMDispenseChain atmDispenser = new ATMDispenseChain();  
        while (true) {  
            int amount = 0;  
            System.out.println("Entrer le montant à avoir");  
            Scanner input = new Scanner(System.in);  
            amount = input.nextInt();  
            if (amount % 10 != 0) {  
                System.out.println("le montant doit être un multiple de 10.");  
                return;  
            }  
            // process the request  
            atmDispenser.c1.dispense(new Currency(amount));  
        }  
    }  
}
```

Exercice 3

Un Thali est un repas [indien](#) et [népalai](#), c'est un assortiment de plats (de l'entrée au dessert) servis généralement dans de petits récipients en métal disposés sur un plateau rond. On voudrais créer différents types de Thali(gujarati, punjabi..etc,pour un restaurant végétarien indien.



Solution exercice 3 : patron factory

+++++++L'interface produit abstraite Thali ++++++=

```
public abstract class Thali {
    public abstract void addSabji();
    public abstract void addDal();
    public abstract void addRice();
    public abstract void addRoti();

    public void makeThali() {
        System.out.println("Veg Thali will be ready in 30 minutes.");
    }
}
```

+++++++implémentation du 1^{er} produit concret GujaratiThali ++++++

```
public class GujaratiThali extends Thali {

    @Override
    public void addSabji() {
        System.out.println("Adding Sabji for Veg Gujarati Thali.");
    }

    @Override
    public void addDal() {
        System.out.println("Adding Dal for Veg Gujarati Thali.");
    }

    @Override
    public void addRice() {
        System.out.println("Adding Rice for Veg Gujarati Thali.");
    }

    @Override
    public void addRoti() {
        System.out.println("Adding Roti for Veg Gujarati Thali.");
    }
}
```

+++++++implémentation du 2eme produit concret PunjabiThali ++++++

```
public class PunjabiThali extends Thali {

    @Override
    public void addSabji() {
        System.out.println("Adding Sabji for Veg Punjabi Thali.");
    }

    @Override
    public void addDal() {
        System.out.println("Adding Dal for Veg Punjabi Thali.");
    }

    @Override
    public void addRice() {
        System.out.println("Adding Rice for Veg Punjabi Thali.");
    }
}
```

```

        @Override
        public void addRoti() {
            System.out.println("Adding Roti for Veg Punjabi Thali.");
        }
    }
}
++++++=implémentaion de la factory abstraite BaseThaliRestaurant ++++++
public abstract class BaseThaliRestaurant {

    public abstract Thali createThali(String type);
}

++++++=implémentation de la factory concrete ThaliRestaurant ++++++
public class ThaliRestaurant extends BaseThaliRestaurant{
    @Override
    public Thali createThali(String type){
        Thali thali;
        switch (type.toLowerCase())
        {
            case "gujarati":
                thali = new GujaratiThali();
                break;
            case "punjabi":
                thali = new PunjabiThali();
                break;

            default: throw new IllegalArgumentException("No such Thali.");
        }

        thali.addSabji();
        thali.addRice();
        thali.addDal();
        thali.addRoti();
        thali.makeThali();
        return thali;
    }
}
++++++=classe test ++++++
public class FactoryDesignPattern {

    public static void main(String args[]){
        BaseThaliRestaurant thalirestaurant = new ThaliRestaurant();
        Thali gujaratiThali = thalirestaurant.createThali("gujarati");
        Thali punjabiThali = thalirestaurant.createThali("punjabi");
    }
}

```

Exercice 4

On veut simuler l'appli météo sur un smartDevice qui envoie la nouvelle température à chaque changement.

Solution Exercice 4 Observer

+++++=le sujet Observable : interface+++++=

```
public interface WeatherSubject {
    public void addObserver(WeatherObserver weatherObserver);
    public void removeObserver(WeatherObserver weatherObserver);
    public void doNotify(); }
```

+++++=observateur interface+++++=

```
public interface WeatherObserver {
    public void doUpdate(int temperature); }
```

+++++=implementation de l'objet observable+++++=

```
public class WeatherStation implements WeatherSubject {

    List<WeatherObserver> weatherObservers;
    int temperature;

    public WeatherStation(int temperature) {

        weatherObservers = new ArrayList<WeatherObserver>();
        this.temperature = temperature; }

    @Override
    public void addObserver(WeatherObserver weatherObserver) {
        weatherObservers.add(weatherObserver); }

    @Override
    public void removeObserver(WeatherObserver weatherObserver) {
        weatherObservers.remove(weatherObserver); }

    @Override
    public void doNotify() {
        for(WeatherObserver weatherObserver : weatherObservers)
            weatherObserver.doUpdate(temperature); }

    public void setTemperature(int newTemperature) {
        System.out.println("\nWeather station setting temperature to " +
            newTemperature);
        temperature = newTemperature;
        doNotify(); } }
```

+++++=observateur concret 1+++++=

```
public class WeatherCustomer1 implements WeatherObserver {
    @Override
    public void doUpdate(int temperature) {
        System.out.println("Weather customer 1 just found out the
        temperature is:" + temperature); } }
```

+++++=observateur concret 2+++++=

```
public class WeatherCustomer2 implements WeatherObserver {
    @Override
    public void doUpdate(int temperature) {
        System.out.println("Weather customer 2 just found out the
        temperature is:" + temperature); } }
```

```

+++++++ classe Test+++++++
public class Demo {
public static void main(String[] args) {
    WeatherStation weatherStation = new WeatherStation(33);

    weatherStation.addObserver(wc2);
    weatherStation.setTemperature(34);

    weatherStation.removeObserver(wc1);

    weatherStation.setTemperature(35);    } }

```

la console affichera
Weather station setting temperature to 34
Weather customer 2 just found out the temperature is:34
Weather customer 1 just found out the temperature is:34
Weather station setting temperature to 35
Weather customer 2 just found out the temperature is:35

Exercice 5

Des utilisateurs s'abonnent à un blog. Lorsqu'un nouveau article est posté, il va envoyer une mise à jour aux utilisateurs enregistrés disant qu'un nouvel article est posté.

Solution exercice 5

```

+++++++observateur
interface+++++=

```

```

public interface Observer {
public void update();
public void setSubject(Subject subject); }

```

```

+++++++observateur concret
+++++=
+

```

```

public class User implements Observer {
private String article;
private Subject blog;
public void setSubject(Subject blog) {

this.blog = blog;
article = "No New Article!"; }

@Override
public void update() {
System.out.println("State change reported
by Subject.");
article = (String) blog.getUpdate(); }

public String getArticle() {
return article; } }

```

```

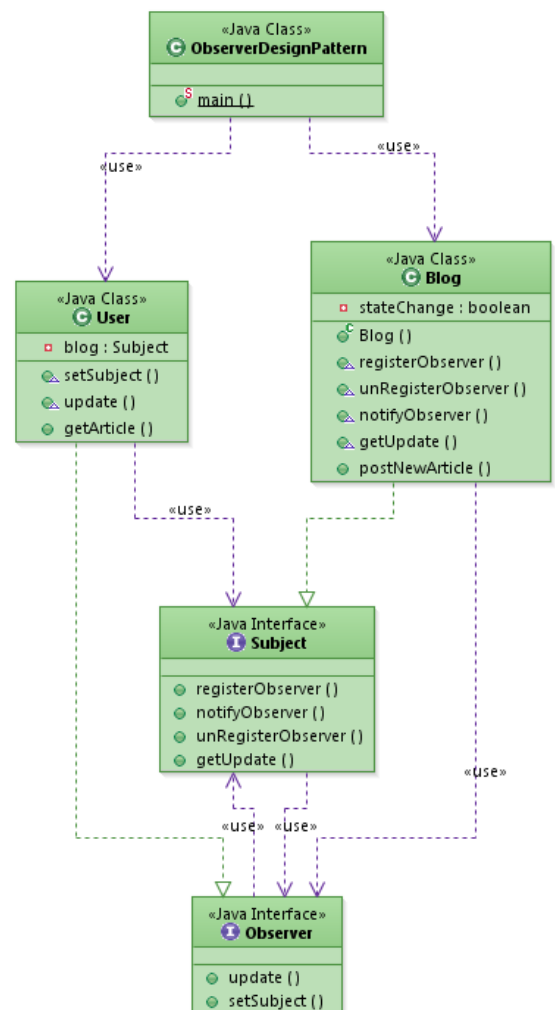
+++++=le sujet Observable : interface
+++++=

```

```

public interface Subject {
public void registerObserver(Observer
observer);
public void notifyObserver();
public void unregisterObserver(Observer
observer);
public Object getUpdate(); }

```



+++++++implementation de l'objet observable+++++++

```
import java.util.ArrayList;
import java.util.List;

public class Blog implements Subject {
    List observersList;
    private boolean stateChange;

    public Blog() {
        this.observersList = new ArrayList();
        stateChange = false;    }

    public void registerObserver(Observer observer) {
        observersList.add(observer);    }
    public void unregisterObserver(Observer observer) {
        observersList.remove(observer);    }
    public void notifyObserver() {
        if (stateChange) {
            for (Observer observer : observersList) {
                observer.update();
            }
        }
    }
    public Object getUpdate() {
        Object changedState = null;
        // should have logic to send the
        // state change classe test to querying observer
        if (stateChange) {
            changedState = "Observer Design Pattern";
            return changedState;    }
    }

    public void postNewArticle() {
        stateChange = true;
        notifyObserver();    } }
```

```
+++++++=
public class ObserverDesignPattern {
    public static void main(String args[]) {
        Blog blog = new Blog();
        User user1 = new User();
        User user2 = new User();

        blog.registerObserver(user1);
        blog.registerObserver(user2);

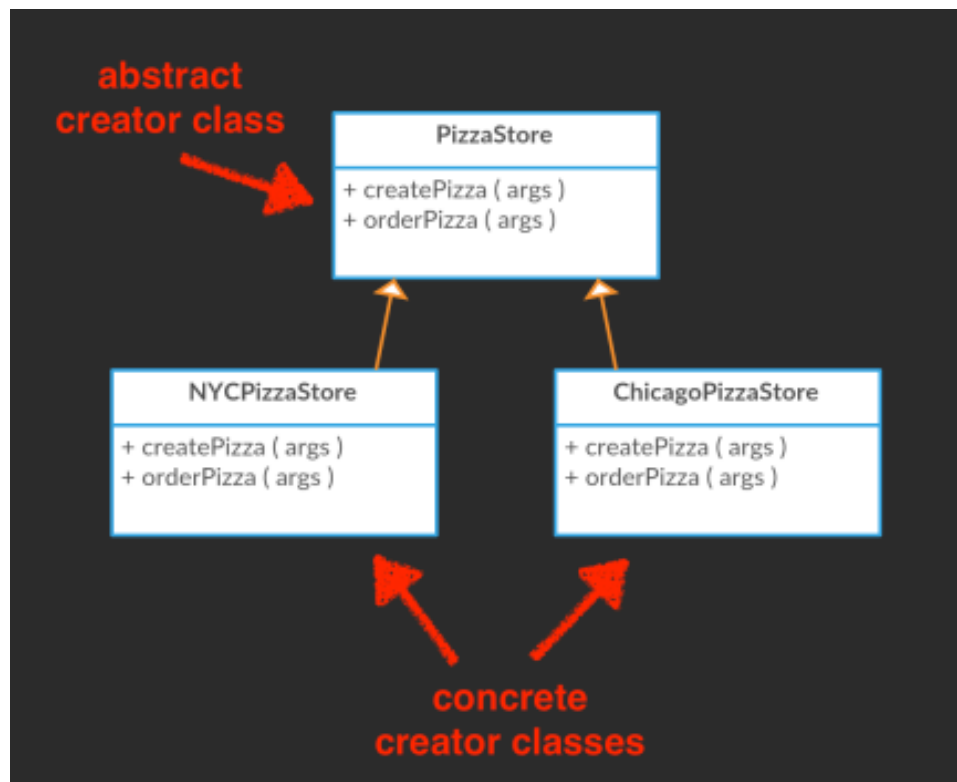
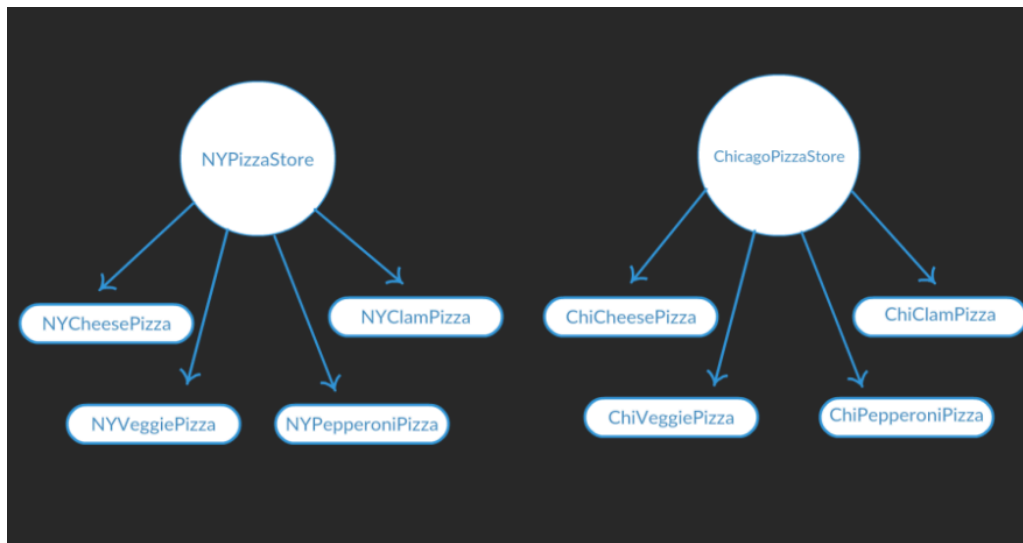
        user1.setSubject(blog);
        user2.setSubject(blog);

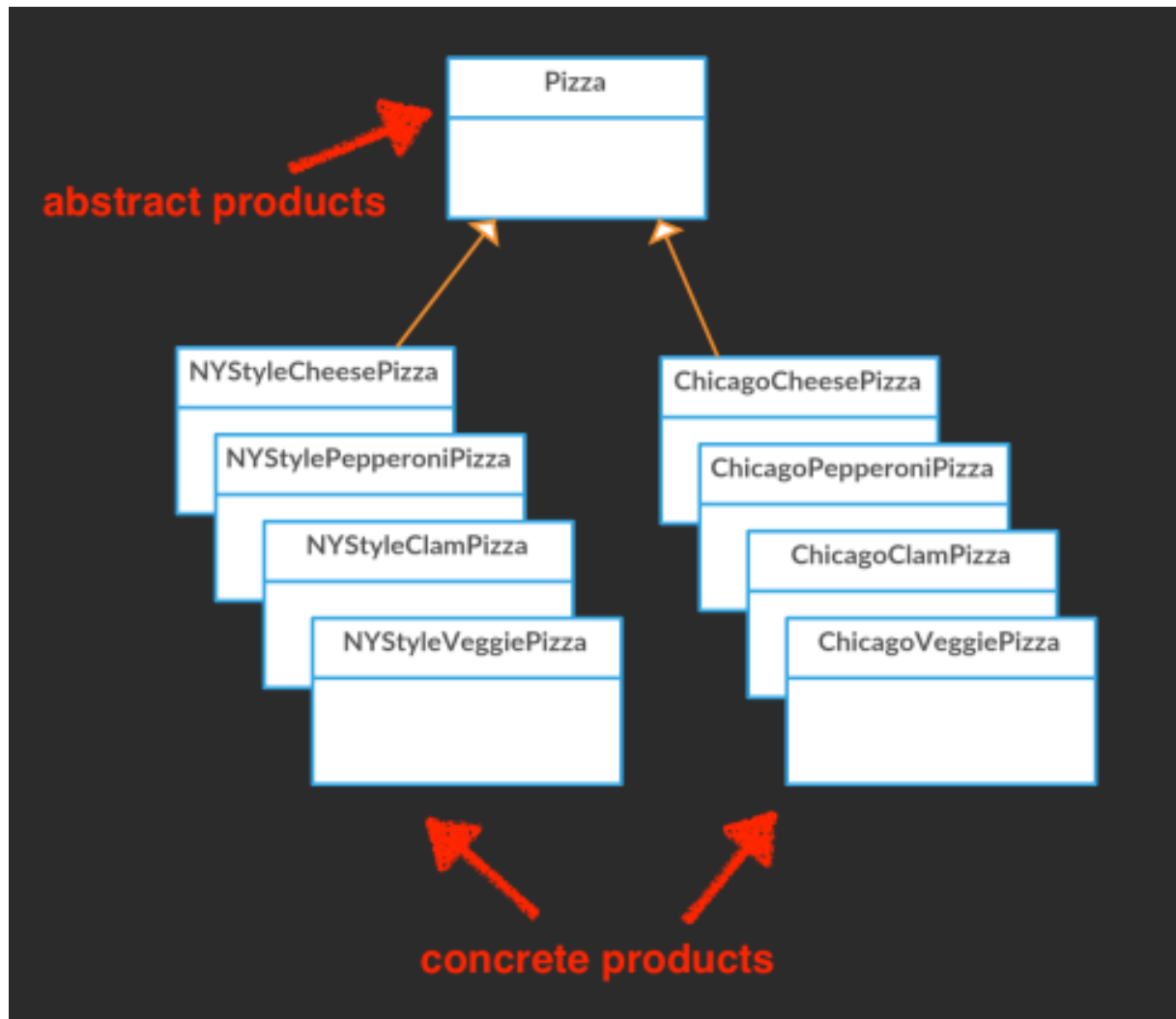
        System.out.println(user1.getArticle());
        blog.postNewArticle();
        System.out.println(user1.getArticle()); } }
```

Exercice 6

Terminer l'exemple du cours pdf Factory(pizza store) de telle manière à ce qu'il implémente le vrai factory method pattern ; on voudrais créer des pizza store dans différents états américains, chacun aura sa propre façon de faire la pizza.

Solution exercise 6





+++++=abstract factory créateur abstrait

```
public abstract class PizzaStore {  
  
    abstract Pizza createPizza(String item);  
  
    public Pizza orderPizza(String type) {  
        Pizza pizza = createPizza(type);  
        System.out.println("— Making a " + pizza.getName() + " —");  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
}
```

++=créateur concret 1++++++

```
public class NYPizzaStore extends PizzaStore {  
  
    Pizza createPizza(String item) {  
        if (item.equals("cheese")) {  
            return new NYStyleCheesePizza();  
        } else if (item.equals("veggie")) {  
            return new NYStyleVeggiePizza();  
        } else if (item.equals("clam")) {  
            return new NYStyleClamPizza();  
        } else if (item.equals("pepperoni")) {  
            return new NYStylePepperoniPizza();  
        } else return null;  
    }  
}
```

+++++=créateur concret 2++++++

```
public class ChicagoPizzaStore extends PizzaStore {  
  
    Pizza createPizza(String item) {  
        if (item.equals("cheese")) {  
            return new ChicagoStyleCheesePizza();  
        } else if (item.equals("veggie")) {  
            return new ChicagoStyleVeggiePizza();  
        } else if (item.equals("clam")) {  
            return new ChicagoStyleClamPizza();  
        } else if (item.equals("pepperoni")) {  
            return new ChicagoStylePepperoniPizza();  
        } else return null;  
    }  
}
```

+++++++produit abstrait+++++++

```
public abstract class Pizza {
    String name;
    String dough;
    String sauce;
    ArrayList<String> toppings = new ArrayList<String>();

    void prepare() {
        System.out.println("Prepare " + name);
        System.out.println("Tossing dough...");
        System.out.println("Adding sauce...");
        System.out.println("Adding toppings: ");
        for (String topping : toppings) {
            System.out.println("    " + topping);
        }
    }

    void bake() {
        System.out.println("Bake for 25 minutes at 350");
    }

    void cut() {
        System.out.println("Cut the pizza into diagonal slices");
    }

    void box() {
        System.out.println("Place pizza in official PizzaStore box");
    }

    public String getName() {
        return name;
    }
}
```

+++++++produit concret 1+++++++

```
public class NYStyleCheesePizza extends Pizza {

    public NYStyleCheesePizza() {
        name = "NY Style Sauce and Cheese Pizza";
        dough = "Thin Crust Dough";
        sauce = "Marinara Sauce";

        toppings.add("Grated Reggiano Cheese");
    }
}
```

++++=produit concret 2+++++=

```
public class ChicagoStyleCheesePizza extends Pizza {

    public ChicagoStyleCheesePizza() {
        name = "Chicago Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";

        toppings.add("Shredded Mozzarella Cheese");
    }

    void cut() {
        System.out.println("Cutting the pizza into square slices");
    }
}
```

++++test Classe

```
public class PizzaTestDrive {

    public static void main(String[] args) {
        PizzaStore nyStore = new NYPizzaStore();
        PizzaStore chicagoStore = new ChicagoPizzaStore();

        Pizza pizza = nyStore.orderPizza("cheese");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("cheese");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");

        pizza = nyStore.orderPizza("clam");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("clam");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");

        pizza = nyStore.orderPizza("pepperoni");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("pepperoni");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");

        pizza = nyStore.orderPizza("veggie");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("veggie");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");
    }
}
```

